

The f1r3score Player

Specification for a Rust tool that reads a textual score and plays a run of it

F1R3FLY.io

Specification — 25 September 2026

Revision 1. Implements *Scores as Processes* (second version), publications/f1r3score

Abstract

This specification defines **f1r3score**, a Rust command-line player and library for the score calculus of *Scores as Processes*. The player reads a score written in a textual surface syntax, elaborates it to a closed term of the calculus, and executes one run: it enumerates candidates at each location, evaluates the graded where-clauses on them, groups them into contention sets, selects a maximal matching with probability proportional to the product of its weights, and stamps the emitted notes with onsets computed from the data. Its outputs are the performance (the multiset of timed notes of positive length), a replayable trace, a Standard MIDI File, and optionally live MIDI.

Three properties organise the design. *Faithfulness*: the engine implements the reduction and resolution rules of the note literally, with no knowledge of voices, fans or idioms; those exist only as library definitions in the surface syntax. *Exactness*: weights are exact rationals and chance is consumed by exact interval decoding, so a score, a chance source and a scheduler determine a run bit for bit on every platform. *Conformance*: the Python reference simulator **skeinsim.py** and its thirteen tests are ported as the acceptance suite, with the same statistics and the same thresholds.

The document fixes the textual format (§3), the crate architecture and data model (§4), the clause language and its checkable fragment (§5), the engine (§6), the chance sources (§7), the outputs and command line (§§8–9), conformance (§10), a phased plan (§11) and the decisions still open (§12).

Contents

1	Purpose and scope	2
1.1	What the tool does	2
1.2	What was reviewed	3
1.3	What the tool is not	3
1.4	Conventions	3
2	The semantic contract	3
3	The textual format	4
3.1	Design position	4
3.2	Declarations	4
3.3	Core process syntax	5
3.4	Definitions: the elaboration-time macro layer	5
3.5	The standard library	6
3.6	Clauses and factors	6
3.7	Twinkle, in full	7
3.8	Diagnostics and lints	7

4	Architecture and data model	7
4.1	Crates	7
4.2	Terms and hash-consing	8
4.3	The soup	8
4.4	Digests	9
5	The clause language	9
5.1	Grammar	9
5.2	Atoms and the candidate view	9
5.3	Tables and machines	10
5.4	Evaluation	10
5.5	The checkable fragment	10
6	The engine	11
6.1	One resolution	11
6.2	Maximal matchings	12
6.3	Scheduling simultaneous sets	12
6.4	Time	12
6.5	Termination and productivity	12
6.6	Collection of inert locations	13
7	Chance sources	13
7.1	One interface	13
7.2	Which source a choice uses	13
7.3	Factorised resolution	14
7.4	Human choice	14
8	Outputs	14
8.1	Performance	14
8.2	Trace	14
8.3	MIDI	15
8.4	Live MIDI	15
9	Command line	15
10	Conformance	16
10.1	The ported suite	16
10.2	Additional tests	16
11	Phasing	17
12	Decisions to confirm	17

1 Purpose and scope

1.1 What the tool does

Given a file `piece.score`, the command

```
f1r3score play piece.score --chance prng:7 --notes 64 --midi piece.mid
```

parses and elaborates the score, runs it until 64 notes of positive length have sounded or nothing more can fire, writes the performance to standard output, and writes a MIDI file. The same run, replayed from its trace, reproduces the same performance exactly.

1.2 What was reviewed

- `publications/f1r3score/score-calculus.tex` (second version): the normative semantics. Section and result numbers below refer to it.
- `publications/f1r3score/skeinsim.py`: the stdlib-only reference simulator (716 lines, tests T0–T12). Its data model — a soup indexed by (quote, timbre), hash-consed quotes, generic candidate generation — is the model this specification adopts.
- `publications/fuzzyware/graded-where-pln.tex` (Paper II): the two formula sorts, per-candidate evaluation, contention sets, and the resolution algebra $(\mathbb{R}_{\geq 0}, +, \times, 0, 1)$.
- `publications/f1r3skein/f1r3skein-drawn-distributions.tex`: the weighted state machines drawn on the instrument, their canonical form, the *Machines* Theory, and exact interval decoding (Algorithm 1).
- `F1R3Games/F1R3Skein`: the Rust workspace (`spigot_stream`, `dual_spigot`, `spigot_midi`, `leap_spigot`), for reusable digit streams and MIDI code.
- The rho-weighted crate and its design (`publications/rho-life/Gillespie`), for the ideal-versus-checkable discipline expressed as types.

1.3 What the tool is not

- **Not an interpreter on a shard.** Following the precedent set for the weighted-GSLT simulator, the player is a separate, offline execution path. It has no dependency on `f1r3node-rust`, `RSpace` or `consensus`.
- **Not F1R3Skein.** The score calculus generalises an idea from the instrument; it is not a formalisation of it. The instrument’s streams are one score among many (Remark 4.6 of the note), and the player treats them so: a drawn machine can be imported as a clause factor, nothing more.
- **Out of scope:** tempo as a graded dimension, dynamics and articulation, timbre change in a communication, complex grading, and learning styles from a corpus — all listed as open threads in the note. The renderer takes a fixed tempo as a rendering parameter only.

1.4 Conventions

MUST, SHOULD and MAY have their RFC 2119 meanings. Requirements are numbered Rn and decisions Dn . Rust identifiers are indicative; the phase exit criteria are normative.

2 The semantic contract

This section restates, as obligations on the implementation, what the note defines. Anything not stated here follows the note.

Requirement 2.1 (Data). A score fixes a finite pitch alphabet Π containing the rest r , a finite duration alphabet Δ with lengths $|\cdot| : \Delta \rightarrow \mathbb{Q}_{\geq 0}$ in whole notes, containing the null duration ϵ with $|\epsilon| = 0$, and a finite set of timbres. Π and Δ MUST be disjoint, since a datum fixes the polarity of its name.

Requirement 2.2 (Terms). Names are variables or triples $\langle @P, \tau, d \rangle$; processes are 0 , receipts for $(y \leftarrow x \text{ where } \psi)P$, joins (chord receipts, Definition 5.5), messages $x!(Q, \tau, d)$, parallel composition and $*x$. There is no replication, no `new`, and no recursion former.

Requirement 2.3 (Congruence). Structural congruence is the least congruence closed under α -conversion making $(|, 0)$ a commutative monoid and satisfying $*\langle @P, \tau, d \rangle \equiv P$. Two names are equivalent when their quoted processes are congruent and their decorations are equal. The implementation **MUST** decide name equivalence exactly.

Requirement 2.4 (Communication). A candidate is a top-level receipt and a top-level message at the same location and timbre, of opposite polarities, whose payload timbre equals the subject timbre ($\text{COMM}_1/\text{COMM}_2$). Firing it emits the note (τ, π, δ) — the pitch from whichever side carries one — and continues with $P\{\langle @Q, \tau, d \rangle/y\}$. Every reduction emits exactly one note; there are no silent steps (Proposition 2.4).

Requirement 2.5 (Weights and resolution). A candidate at a receipt with clause ψ is *available* iff $\psi\llbracket c \rrbracket > 0$. Contention sets are the connected components of the relation “shares a receipt or a message” over available candidates. A set is resolved by choosing a maximal matching $\mathcal{M}$ with probability proportional to $\prod_{c \in \mathcal{M}} \psi_c\llbracket c \rrbracket$ and firing all of it (Definition 3.3).

Requirement 2.6 (Time). Every top-level receipt and message carries a stamp in $\mathbb{Q}_{\geq 0}$. A communication has onset $\max(t_r, t_m)$, emits the timed note (t, τ, π, δ) , and stamps what its continuation brings to top level with $t + |\delta|$; a chord stamps with $t + \max_i |\delta_i|$. Runs are onset-ordered: a set is resolved only when its onset (the least onset of its candidates) is minimal among all sets. The performance is the multiset of timed notes of positive length.

Requirement 2.7 (Engine neutrality). The engine **MUST** know nothing of voices, fans, hands, servers, machines or idioms. Those are definitions in the surface syntax that elaborate to terms. (This is the property `skeinsim.py` states in its header, and it is what makes Finding 8.3 — the reflective voice plays the notes of the step semantics — a test of the calculus rather than of the code.)

Remark 2.8 (What the note leaves to the implementation). Three things are parameters of the semantics rather than parts of it: the chance source (Definition 3.3 names it a parameter), the order in which simultaneously due contention sets are resolved, and whether a set fires a maximal matching or a single candidate. The last is `mode='one'` in the simulator and is retained only as a diagnostic (T3). The first two are made explicit and recorded in every trace (§7, §6.3).

3 The textual format

3.1 Design position

Decision 3.1 (A dialect of the `f1r3—@ng` control flow sublanguage). The surface syntax reuses the concrete syntax of the `f1r3—@ng` CFL wherever the calculus has the same former: `for (y <- x where ...) {P }, x!(...), |, *x, @P`. What is new is the decoration of names, the alphabet declarations, and a small elaboration-time macro layer. The grammar is kept small enough to be restated later as a **Theory Score** in the `f1r3—@ng` DDL once the Theory syntax lands in the node; the parser crate is therefore a replaceable front end over a stable core AST (§4).

Files use the extension `.score` and are UTF-8. Comments are `//` to end of line and `/* ... */`.

3.2 Declarations

A score file is a sequence of imports, declarations and definitions, followed by exactly one `play` item giving the initial configuration.

```

score Twinkle

pitches    { r, C3, F3, G3, C4, D4, E4, F4, G4, A4 }    // r: the rest, required
durations  { q = 1/4, h = 1/2 }                        // eps = 0 is implicit
timbres    { piano = gm(0) on 1, bass = gm(32) on 2 }  // GM program, MIDI channel

```

- Requirement 3.2** (Alphabets). (i) `pitches` lists Π *in order*; that order defines `step(k)` (§5.2). `r` MUST be present and is unordered. A pitch written in scientific notation (`C4`, `F#3`, `Bb2`) gets its MIDI number implicitly (`C4 = 60`); any other identifier MUST be given one, `deg7 = 67`.
- (ii) `durations` gives each length as a non-negative rational literal (`1/6` for a triplet eighth, `3/8` for a dotted quarter). `eps = 0` is always present and MUST not be redeclared. Other zero-length durations are permitted.
- (iii) Ranges MAY abbreviate alphabets: `pitches {r, scale major C4 .. C7 }`, `durations {w=1, h=1/2, q=1/4, e=1/8, s=1/16 }`. Expansion is at elaboration time; the alphabet is always finite.
- (iv) Identifiers in Π , Δ and the timbre set MUST be pairwise distinct; the elaborator rejects overlap.

3.3 Core process syntax

calculus	surface	note
0	0	
$\langle @P, \tau, d \rangle$	<code><@P, tau, d></code>	parenthesise compound P
$x!(Q, \tau, d)$	<code>x!(Q, tau, d)</code>	
$\text{for}(y \leftarrow x \text{ where } \psi)P$	<code>for (y <- x where psi) {P }</code>	clause optional
<code>join</code>	<code>for (y1 <- x1 & y2 <- x2 where psi) {P}</code>	chord
$P \mid Q$	<code>P Q</code>	
$*x$	<code>*x</code>	
<code>stamp</code>	<code>at 3/4 {P }</code>	top level of play only

The underscore `_` is a binder that is not used. In the `play` item every top-level receipt and message is stamped 0 unless an `at` says otherwise (an entrance).

3.4 Definitions: the elaboration-time macro layer

The note builds everything from abbreviations: `N`, `Rec`, `Fan`, `Step`, `Server`, `Voice`, `Call`, `Resp`. The surface syntax needs exactly this facility and no more.

```

def N(L: proc, tau: timbre, p: pitch, d: dur; K: proc) =
  for (_ <- <@L, tau, p>) { K } | <@L, tau, d>!(0, tau, r)

```

- Requirement 3.3** (Definitions are macros). (i) Parameters are sorted: `proc`, `name`, `timbre`, `pitch`, `dur`, `clause`, and `cont` (a continuation: a definition of arity one in a name, passed by name).
- (ii) Definitions are expanded at elaboration time and MUST form an acyclic call graph. A recursive definition is rejected with a diagnostic pointing to §4.2 of the note: unbounded behaviour is obtained by reflection, not by recursion (Remark 2.1).
- (iii) Expansion is capture-avoiding. The elaborated term is a closed term of the core syntax and nothing else.

Finite products. $\prod_{v \in \Delta^+} \prod_{t \in \Pi}$ is written as a comprehension that expands to a parallel composition:

```
def Fan(x: name, tau: timbre) =
  par v in durations+, t in pitches {
    <@*x, tau, v>!( <@*x, tau, v>!( *x, tau, t ), tau, t )
  }
```

Generators range over `pitches`, `pitches-r` (the ordered pitches), `durations`, `durations+` (positive length), and explicit lists.

Lines. A written part is a right-nested chain of written notes. The derived form

```
line(Lm, piano) [ C4 q, C4 q, G4 q, G4 q, A4 q, A4 q, G4 h ]
```

elaborates to $N_{Lm, \text{piano}}(C4, q; \dots N_{Lm, \text{piano}}(G4, h; 0) \dots)$. An optional trailing `then K` replaces the final 0 with a continuation, which is how a written phrase hands off to a responder (§6.5 of the note).

Locations. A *base* is written `base "Lm"` and elaborates to the closed receipt `for(_ ← <@0,  $\tau_\perp$ , r>) 0` tagged with the string, where $\tau_\perp$ is a reserved timbre no message may use. Distinct tags give inequivalent bases.

3.5 The standard library

Requirement 3.4 (`std.score`). The distribution **MUST** ship `std.score`, written entirely in the surface syntax, defining: `N`, `line`; `Rec`, `Fan`, `Seed` (H_0); `Step`, `Req`, `Server`, `Refresh`, `Voice` (§4.2 of the note); `Call`, `Resp` (handoff); `HocketA`/`HocketB` (alternation); and the dual (rhythm-leads) variants of `Fan` and `Step`. `Voice` **MUST** be the reflective voice, with no engine support: its server keeps its own code on a code channel and re-sends it (Proposition 4.4).

```
// std.score, excerpt: the reflective voice
def Req(z: name, tau: timbre, BS: proc) = <@BS, tau, eps>!( *z, tau, r )

def Server(tau: timbre, psi: clause, BS: proc, BC: proc) =
  for (w <- <@BS, tau, r>) {
    Step(w, tau, psi, Req(_, tau, BS)) | Refresh(tau, BC)
  }

def Refresh(tau: timbre, BC: proc) =
  for (c <- <@BC, tau, r>) { *c | <@BC, tau, eps>!( *c, tau, r ) }

def Voice(tau: timbre, psi: clause, p0: pitch, u0: dur, tag: string) =
  Server(tau, psi, base tag ++ "/S", base tag ++ "/C")
  | <@(base tag ++ "/C"), tau, eps>!( Server(tau, psi, base tag ++ "/S",
                                          base tag ++ "/C"), tau, r )
  | <@(base tag ++ "/S"), tau, eps>!( Seed(base tag, tau, p0, u0), tau, r )
```

3.6 Clauses and factors

Clause syntax is given in §5. At the top level a score may name factors so that style, register and physicality are written once and combined:

```
factor stepwise = table step { -2..2: 3, _: 3/10 }
factor country  = machine pitch { C4 -> D4 @ 2, C4 -> E4 @ 3, D4 -> E4 @ 1, ... }
factor rhythm   = machine dur uniform
```

```
play Voice(piano, country * stepwise * rhythm, C4, q, "A")
  | Voice(bass, walking * rhythm, C3, h, "B")
```

3.7 Twinkle, in full

```
score Twinkle
import std
pitches { r, C3, F3, G3, C4, D4, E4, F4, G4, A4 }
durations { q = 1/4, h = 1/2 }
timbres { piano = gm(0) on 1, bass = gm(32) on 2 }

play line(base "Lm", piano) [ C4 q, C4 q, G4 q, G4 q, A4 q, A4 q, G4 h,
                             F4 q, F4 q, E4 q, E4 q, D4 q, D4 q, C4 h ]
  | line(base "Lb", bass) [ C3 h, C3 h, F3 h, C3 h, F3 h, C3 h, G3 h, C3 h ]
```

Requirement 3.5 (Golden score). This file MUST play, under every scheduler, the 22 timed notes of the table in §2.5 of the note, ending at 4 whole notes (conformance test T12).

3.8 Diagnostics and lints

The elaborator MUST reject: overlapping alphabets; undeclared data or timbres; recursive definitions; ill-sorted arguments; use of $\tau_{\perp}$; clauses outside the checkable fragment (§5.5). It SHOULD warn when a message’s payload timbre differs from its subject’s timbre (such a message can never communicate), and when a top-level receipt and message of the same polarity share a location (they never communicate, Proposition 2.4(ii)).

4 Architecture and data model

4.1 Crates

crate	responsibility
score-core	data alphabets, hash-consed terms, congruence normal form, substitution, names and locations. No I/O.
score-syntax	lexer, parser, surface AST, macro elaboration, std.score , pretty-printer, diagnostics with spans.
score-logic	crisp and graded formula ASTs, the checkable-fragment type, candidate views, evaluation, tables and machines.
score-engine	the soup, candidate generation, contention sets, maximal matchings, resolution, time, scheduling, collection, trace events.
score-chance	chance sources: PRNG digit streams, spigot digit streams, exact interval decoding, human choice, replay.
score-render	performance and trace serialisation, Standard MIDI File writer, live MIDI (feature live).
f1r3score	the binary: command line, wiring, exit codes.
score-conformance	the ported skeinsim suite and golden files.

Requirement 4.1 (Portability). **score-core**, **score-syntax**, **score-logic**, **score-engine** and **score-chance** (without the spigot feature) MUST build for **wasm32-unknown-unknown** and MUST perform no I/O, so that the same engine can be embedded in a browser page or an Apple Vision Pro client. They SHOULD have no dependencies beyond **num-bigint** / **num-rational**. The workspace targets stable Rust, edition 2021, with **rust-version** pinned.

4.2 Terms and hash-consing

```

pub struct Datum(u16);           // index into Pi + Delta (disjoint)
pub enum Polarity { Channel, CoChannel }
pub struct Timbre(u16);
pub struct ProcId(u32);          // hash-consed, congruence-normal process
pub struct Loc { pub quote: ProcId, pub timbre: Timbre }

pub enum Name { Var(VarIdx), Quote { proc: ProcId, timbre: Timbre, datum: Datum } }

pub enum Proc {
  Nil,
  Par(Box<[ProcId]>),             // flattened, >= 2, sorted by id, no Nil
  Recv { binds: Box<[Name /*subject*/]>, clause: ClauseId, body: ProcId },
  Send { subj: Name, payload: ProcId, ptimbre: Timbre, carry: Datum },
  Drop(Name),                    // only on a variable: *<@P,...> normalises
  to P
}

pub struct Arena { /* Vec<Proc>, HashMap<Proc, ProcId> */ }

```

Requirement 4.2 (Congruence by construction). The arena MUST intern only normal forms: parallel compositions flattened, 0 removed, children sorted by `ProcId` (a multiset), singletons unwrapped; $\langle @P, \tau, d \rangle$ replaced by P ; bound variables in de Bruijn form so that α -equivalent terms are identical. Then $P \equiv Q$ iff their ids are equal, and location equivalence is integer equality. A property test MUST check this against an independent, unoptimised congruence decision procedure on random terms.

Remark 4.3 (Why sorting by id is sound). Children are interned before their parent, so sorting by id is a canonical order on already-normal children. Ids depend on interning order, which differs between runs of different scores; they are never serialised as identities. Serialised terms and digests use a structural encoding (§4.4).

Requirement 4.4 (Substitution). Firing substitutes a closed name $\langle @Q, \tau, d \rangle$ for the bound variable and re-normalises; occurrences of $*y$ become Q and names built from y are compared up to congruence thereafter. Substitution MUST be memoised on (body, name) since the same body is instantiated repeatedly by a voice.

4.3 The soup

The running configuration is not stored as a term. As in the simulator, it is a *soup*: the top-level receipts and messages, each an *occurrence* with a stable occurrence number, a stamp and (for receipts) its clause, indexed by location.

```

pub struct RecvOcc { id: OccId, subjects: Box<[Name]>, clause: ClauseId,
  body: ProcId, stamp: Time, stream: StreamKey }
pub struct SendOcc { id: OccId, subj: Name, payload: ProcId,
  ptimbre: Timbre, carry: Datum, stamp: Time }

pub struct Soup {
  recv: BTreeMap<Loc, Vec<RecvOcc>>,
  send: BTreeMap<Loc, Vec<SendOcc>>,
  dirty: BTreeSet<Loc>,           // locations whose candidates may have
  changed
  next_occ: OccId,
}

```

Requirement 4.5 (Determinism of iteration). Every collection whose iteration order can influence a run **MUST** be ordered (**BTreeMap**, sorted **Vec**), keyed by occurrence number or by a structural key. Occurrence numbers are assigned in firing order. Consequently a score, a chance source and a scheduler determine the run exactly.

Putting a process into the soup (*spawning* at stamp t) decomposes its normal form: **Par** spawns each child; **Recv** and **Send** become occurrences at their subjects' locations; **Nil** does nothing. A top-level $*y$ cannot occur in a closed term.

4.4 Digests

Every elaborated score has a *digest*: SHA-256 over a canonical, id-independent serialisation of its term, alphabets and lengths. The digest heads every trace and performance file and is the identity under which a score would later be stored as a content-addressed value on a shard.

5 The clause language

5.1 Grammar

The two sorts of Paper II, with the atoms of Definition 3.1 of the note.

sort	surface	meaning
graded ψ	3/2, 0.3	constant $w \in \mathbb{Q}_{\geq 0}$
	[beta]	$[\beta]$
	psi * psi, psi + psi	$\otimes, \oplus$
	<>psi	graded behavioural modality $\langle K \rangle_v$
	mu X. psi, nu X. psi	graded fixed points
crisp β	table key {... }, machine ...	sugar, below
	factor name	reference to a named factor
	true, !b, b && b, b b	propositional
spatial σ	atoms	§5.2
	<>b, leads(b), nu X. b	behavioural
	true, 0, s s, !s, s && s	over processes
	<n, tau, d>!(s, tau, d)	out: a message
	for(<n, tau, d>) s	in: a receipt
	@s	name predicate

In message and receipt patterns, $_$ matches any location, timbre or datum; **n** is itself a spatial formula applied to the quoted location, so patterns nest. Graded formulae have no negation (Paper II, Definition 2.4).

5.2 Atoms and the candidate view

A clause is evaluated on a candidate c against the configuration in which it stands. The view is

```
pub struct View<'a> {
  pub timbre: Timbre, pub pitch: Datum, pub dur: Datum, // the note nu(c)
  pub carry: Datum, // kappa(c)
  pub loc: ProcId, // @S
  pub passes: ProcId, // Q of <@Q,tau,d>
  pub chord: &'a [ChordSlot], // joins only
  pub residue: Lazy<'a, Residue>, // behavioural atoms
  only
}
```

atom	holds when
<code>pitch(s), dur(v), carry(e)</code>	the note's pitch, its duration, the carried datum equal the argument
<code>step(k)</code>	carry and pitch are ordered pitches and $\text{ord}(\kappa) - \text{ord}(\pi) = k$
<code>at(s)</code>	the location's quote satisfies σ
<code>passes(s)</code>	the passed process satisfies σ
<code>held(t)</code>	<code>at(<_,_,>!(true, _, t))</code> (derived)
<code>prev(u)</code>	<code>at(<_,_,u>!true)</code> (derived)
<code>back(j, e)</code>	the record j levels into the past carries e (derived)
<code>last[m1 .. mk]</code>	<code>at(Last(m))</code> , §6.1 of the note (derived)
<code>pitch#i, dur#i, carry#i</code>	per-pattern atoms of a chord receipt
<code>leads(b)</code>	the residue of firing c , in the present configuration, satisfies β

Requirement 5.1 (Derived atoms are sugar). `held`, `prev`, `back`, `last` MUST elaborate to spatial formulae. The evaluator MAY compile them to direct walks over the record chain (as the simulator's `back` and `repeat` do), but a test MUST check the compiled and the formula forms agree on random views, exactly as T9 does for tables.

5.3 Tables and machines

`table key {k: w, ..., _: w }` is a sum over mutually exclusive crisp conjunctions evaluated by lookup, keyed by (`pitch`, `carry`), (`prev`, `dur`), `step`, or any tuple of atoms that are functions of the view. `_` is the default (absent means 0). Ranges such as `-2..2` are allowed on `step`.

`machine pitch {s -> t @ w, ... }` and `machine dur {u -> v @ w, ... }` are Example 5.2's factors: the pitch machine is a table keyed by (`pitch`, `carry`), the duration machine a table keyed by (`prev`, `dur`). The arrow syntax is that of the **Machines** Theory in the drawn-distributions design, and `machine pitch import "file.machine"` reads a canonical machine exported by the instrument, mapping its state indices onto the declared pitch alphabet by position (with the rest as the last state, as on the instrument). `machine dur uniform` is the complete uniform machine.

5.4 Evaluation

Requirement 5.2 (Exact weights). Clause values are exact non-negative rationals (`BigRational`, with a fast path for `u64/u64` that promotes on overflow). Decimal literals are read exactly (`0.3` is $3/10$). No floating point enters a weight, a normalisation, or a choice.

Requirement 5.3 (Purity). Evaluation is a pure function of the view and the configuration. Evaluating a clause on a candidate that does not fire changes nothing.

$\otimes$ short-circuits on 0; $\oplus$ sums; $\lceil \beta \rceil$ is 1 or 0. Graded fixed points are evaluated only inside the checkable fragment, by iteration to the exact fixed point over a finite state space; if the iteration does not reach an exact fixed point within the declared bound the score is rejected at elaboration, never approximated at run time.

5.5 The checkable fragment

The ideal logic defines; the fragment decides. As in **rho-weighted**, the distinction is a type, not a comment.

```

pub struct Checked<F> { formula: F, class: Locality, depth: usize }
pub enum Locality { Local, Behavioural { window: usize } }
impl<F> Checked<F> { pub fn try_new(f: F, alph: &Alphabets) -> Result<Self, WhyNot>;
}

```

Requirement 5.4 (Fragment membership). A clause is admitted iff it is in one of two classes.

- (i) **Local**: no behavioural modality, no **leads**. Its value depends only on the view. Spatial formulae are decidable on the finite quoted terms involved; separating conjunction is decided by enumerating splits of a multiset and **MUST** be bounded by a configured maximum split count.
- (ii) **Window-behavioural**: behavioural modalities and fixed points are admitted when every spatial formula in *every* clause of the score reads the past to depth at most d (the depth grading of the rho-life notes) and the alphabets are finite. Then the state relevant to the formula is finite (§6.2 of the note), and ν is computed exactly by Knaster–Tarski iteration over the abstract graph whose nodes are configurations with every location truncated to depth d and stamps erased.

Anything else is rejected with **WhyNot** naming the offending subformula.

Decision 5.5 (Behavioural modalities read the actual company). In Proposition 6.3 the contexts K range over “the one-hole contexts supplied by the rest of the score”. A player has exactly one such context at each moment: the rest of the current configuration. The player evaluates $\langle K \rangle$ and **leads** against that, and the $\langle \rangle$ surface form carries no explicit context. Quantifying over hypothetical companies is an analysis task, not a playing task, and is out of scope.

Requirement 5.6 (Locality classes drive caching). Local clauses depend only on the candidate, so a location’s candidate list is recomputed only when that location is dirty. If any receipt in the soup carries a behavioural clause, every such receipt’s candidates are re-evaluated after every resolution. The engine **MUST** report in the trace header which regime the run used.

6 The engine

6.1 One resolution

1. **Candidates**. For each dirty location, pair every receipt with every message of opposite polarity whose payload timbre equals the location’s timbre (for a join, every injective assignment of messages to its patterns, all at that location). Build the view, evaluate the clause, keep the candidates of positive weight.
2. **Contention sets**. Union–find over shared receipt or message occurrence numbers, per location. Since communication requires equal locations, a contention set never spans two locations; the engine **MUST** assert this.
3. **Onsets**. A candidate’s onset is $\max(t_r, t_m)$ (for a join, the maximum over all its stamps); a set’s onset is the least of its candidates’.
4. **Due sets**. Let $t_{\min}$ be the least onset over all sets. The due sets are those with onset $t_{\min}$; the scheduler picks one (§6.3).
5. **Alternatives**. Enumerate the maximal matchings of the chosen set in canonical order, each with weight $\prod_{c \in \mathcal{M}} \psi[c]$.
6. **Choice**. Ask the set’s chance source for an index with probability proportional to the weights (§7).

7. **Firing.** For each candidate in the matching: remove both occurrences, emit the timed note, form the substituted continuation and spawn it at the new stamp. Mark affected locations dirty. Append one trace event.

Requirement 6.1 (Canonical order of alternatives). Candidates are ordered by (receipt occurrence number, message occurrence numbers); matchings by the lexicographic order of their sorted candidate lists. Together with R4.5, this makes “the i -th alternative” a well-defined, replayable quantity.

6.2 Maximal matchings

Requirement 6.2 (Stars and the general case). (i) **Star fast path.** If every candidate in a set shares one receipt, or every candidate shares one message, the maximal matchings are the single candidates. This covers every voice, every chord receipt and every null note, and MUST be $O(\text{set size})$ with no enumeration.

(ii) **General case.** Otherwise enumerate maximal matchings of the bipartite (for joins, hypergraph) contention structure by backtracking with maximality pruning. The number of alternatives MUST be bounded by a configurable limit (default 10^5); exceeding it is a run-time error naming the location, never a silent truncation.

(iii) **Mode one.** A diagnostic mode, off by default, in which a set fires a single candidate chosen with probability proportional to its weight. It exists to reproduce T3’s second column and is recorded in the trace header.

6.3 Scheduling simultaneous sets

When several sets are due at the same onset, the note is indifferent for independent parts (Theorem 2.6, Theorem 7.1) and the order matters only where parts contend. The player makes the order an explicit, recorded parameter.

Decision 6.3 (Schedulers). **canonical** (default): the due set whose location has the least structural key (digest of the quote, then timbre). **random:SEED**: a dedicated PRNG, separate from every chance source, shuffles the due sets. **by-timbre:A,B,...**: a stated priority, for the Twinkle-style “melody first / bass first” tests. The scheduler never consumes digits from a musical chance source.

6.4 Time

Stamps are exact rationals. Continuations spawned by a firing receive the stamp of R2.6. Joins use the “longest duration” convention of Definition 5.5; since the note lists this as an open thread, the convention is a single function in the engine, not a scattered assumption.

6.5 Termination and productivity

A run stops at the first of: no available candidate anywhere (the score has ended or every part has fallen silent); a **--notes N** limit on notes of positive length; a **--until T** limit on onsets (whole notes); a **--max-steps** limit on resolutions.

Requirement 6.4 (Unproductive loops). The note does not yet give a syntactic productivity check (open thread 6). The engine MUST therefore bound the number of consecutive zero-length notes at one onset (default 10^5) and stop with an **unproductive** error naming the locations involved. A voice plays exactly two null notes per note (Proposition 4.4), so the bound is never approached by a well-formed voice.

6.6 Collection of inert locations

A voice leaves $|\Delta^+| \cdot |\Pi| - 1$ inert messages and $|\Pi| - 1$ inert hands behind at every note (Findings 8.1 and 8.3). Keeping them is correct but costs memory linear in the length of the piece.

Requirement 6.5 (Freshness collection). Collection is off by default. With `--gc freshness`, after a resolution at a location whose quote is a message (a record, not a base) and which then has no available candidate, the location’s occurrences are dropped and its key is added to a tombstone set. If anything is later spawned at a tombstoned location, the engine **MUST** stop with a **freshness-violated** error: the collection was unsound for this score. For scores built from `std.score` voices this cannot happen (Lemma 4.5).

Remark 6.6 (Why not reachability). Collecting a location because no live term mentions it is not sound in this calculus: any process can quote any closed term it can build, and bases are built from nothing. Freshness is a theorem about how voices use locations, so the player relies on it only where it is checked.

7 Chance sources

7.1 One interface

```
pub trait Chance {
  /// Choose an index into 'weights' (exact, all > 0) with probability
  /// proportional to weight. Returns the index and what was consumed.
  fn choose(&mut self, weights: &[Weight], ctx: &ChoiceCtx) -> (usize, Consumed);
  fn describe(&self) -> SourceSpec;           // recorded in the trace header
}
pub trait DigitStream { fn base(&self) -> u32; fn next(&mut self) -> u32; }
```

Decision 7.1 (Every automatic source is a digit stream). Every non-human source is a stream of digits in some base, and every choice is made by the exact interval decoding of the drawn-distributions design (Algorithm 1): cumulative weights are cleared to a common denominator, digits narrow a b -adic interval, and the choice is the bin that contains it. There is no floating point and no rejection sampling, so a given digit stream yields the same choices on every machine. $K_{\max}$ (default 64 digits per choice) and its deterministic fallback are part of the source’s specification.

spec	source
<code>prng:SEED</code>	a specified counter-based generator (SplitMix64 feeding xoshiro256**, fixed in this specification so it can never drift with a dependency), emitting base-2 ³² digits
<code>spigot:C/B</code>	digits of the fractional part of constant $C \in \{\text{pi}, \text{e}, \text{ln2}, \text{liouville}, \text{champernowne}, \text{thue-morse}\}$ in base B, from <code>spigot_stream</code> (feature <code>spigot</code>); the integer part is skipped
<code>human</code>	an interactive chooser (§7.4)
<code>replay:FILE</code>	the choice indices recorded in a trace

7.2 Which source a choice uses

Theorem 7.1(iii) holds *surely* only when each voice has its own source (T2 uses one stream per voice). The player routes choices through *stream keys*.

Decision 7.2 (Stream keys). Every receipt occurrence carries a stream key, inherited from the receipt that spawned it. The key is the receipt’s timbre by default, and MAY be set explicitly with an annotation `for #A (...)` or on a definition application `Voice(...) #A`; annotations are metadata and do not change the elaborated term’s congruence class or digest. The command line maps keys to sources: `--chance prng:7 --stream A=spigot:pi/16 --stream B=prng:12`. A contention set uses the stream of its least receipt occurrence. Sets with a single alternative (every null note) consume nothing.

7.3 Factorised resolution

Lemma 5.3 (factorised resolution) lets a star whose weights factor as $f(a)g(b)$ be resolved by two independent draws. T4 depends on this: pitch from the hexadecimal digits of π , duration from the base-5 digits of e , one digit each.

Requirement 7.3 (Factor routing). When a star’s clause is a product of factors that are functions of disjoint parts of the view (the elaborator can see this syntactically: a (pitch, carry) machine times a (prev, dur) machine), the engine MAY resolve it factor by factor, each factor drawing from its own stream (`--stream A.pitch=spigot:pi/16 --stream A.dur=spigot:e/5`). This is an equivalent resolution by Lemma 5.3 and MUST be recorded in the trace. Without a per-factor stream the joint draw is used.

7.4 Human choice

Proposition 6.4 makes the improviser a chance source that ignores the probabilities and chooses among available candidates.

Requirement 7.4 (Human chooser). The **human** source presents, for each set with more than one alternative, the alternatives as notes (pitch, duration, carried pitch), with their normalised probabilities shown for guidance, and accepts an index. Only alternatives of positive weight are offered, so every trace a human produces is admissible, and with a viability factor never runs out (Proposition 6.4(iii)). Mixed performance — a human on one stream key, an algorithmic source on the others — is the default when **human** is bound to a single key. In this version the human chooser is step-wise, not real time.

8 Outputs

8.1 Performance

The performance is printed as a table by default and as JSON with `--out perf.json`:

```
{ "score": "sha256:...", "end": "4",
  "notes": [ { "onset": "0", "timbre": "piano", "pitch": "C4", "dur": "q", "len":
    "1/4" },
    { "onset": "0", "timbre": "bass", "pitch": "C3", "dur": "h", "len":
    "1/2" }, ... ] }
```

Notes are sorted by (onset, timbre, pitch, duration) so that two runs with the same performance produce byte-identical files; this is how interleaving invariance is tested. Null notes are excluded (they are not part of the performance); `--include-null` adds them for diagnosis.

8.2 Trace

Requirement 8.1 (Replayable trace). With `--trace run.jsonl` the player writes JSON Lines: a header (format version, player version, score digest, alphabets, every source spec,

stream bindings, scheduler, matching mode, locality regime, collection mode), then one event per resolution: step number, onset, location key, set size, number of alternatives, chosen index, digits consumed per stream, and the notes emitted. `f1r3score replay` MUST reproduce the performance and the event sequence from the score and the trace alone, and MUST fail if the score's digest differs from the header's.

8.3 MIDI

Requirement 8.2 (Standard MIDI File). `--midi out.mid` writes a format-1 file with one track per timbre, a program change from the timbre's declaration, and note-on/note-off pairs at $\text{ticks}(t)$ and $\text{ticks}(t + |\delta|)$. Rests produce no events. The division (ticks per quarter) is the least value making every onset and length in the performance an integer number of ticks, if that is at most 32767; otherwise 960 with rounding and a warning. Tempo is `--bpm` (default 120), a rendering parameter only.

Finding 8.3 (`spigot_midi` cannot be reused as is). `spigot_midi::MidiTrack` writes format 0 from a sequence of (pitch, duration, velocity) notes with no onsets: it is monophonic and sequential by construction. A score is polyphonic, with explicit onsets and simultaneous strikes. The player therefore needs its own small writer. It SHOULD follow `spigot_midi`'s dependency-free style, reuse its `GeneralMidi` enumeration for program names, and MAY later be upstreamed to replace it.

8.4 Live MIDI

With feature `live` and `--live [PORT]`, a playback thread receives timed notes from the engine through a bounded channel and sends them to a MIDI port via `midir`, as `leap_spigot::player` does. The engine runs ahead of the clock by a configurable look-ahead in whole notes. The engine never waits on the clock: live output is a rendering of the same run, and its trace is identical to an offline run's.

9 Command line

```
f1r3score check   piece.score           # parse, elaborate, lint, report
                  fragment and digest
f1r3score expand  piece.score           # print the elaborated core term
f1r3score play    piece.score [options] # run once
f1r3score replay  piece.score run.jsonl [--midi out.mid]
f1r3score machine import skein.machine --pitches piece.score # print as a factor

options for play:
--chance SPEC          default source (prng:SEED | spigot:C/B | human)
--stream KEY=SPEC      bind a stream key (repeatable; KEY.pitch / KEY.dur for
                        factors)
--scheduler S          canonical | random:SEED | by-timbre:A,B,...
--notes N | --until T | --max-steps N
--mode max|one         matching mode (default max)
--gc off|freshness     collection (default off)
--max-alternatives N   bound on maximal matchings per set (default 100000)
--out FILE --trace FILE --midi FILE --bpm N --live [PORT] --include-null
```

Exit codes: 0 success; 1 usage; 2 parse or elaboration error; 3 run-time error (unproductive, freshness-violated, alternative limit); 4 replay mismatch.

10 Conformance

10.1 The ported suite

Each skeinsim test becomes a Rust integration test in `score-conformance`, reading its score from a `.score` file (not constructed in Rust), so the suite tests the front end as well as the engine. Statistical thresholds are those the note reports; tests are seeded and deterministic.

test	what	pass criterion
T0	inertness of the unplayed	one live location per step; exactly 79 stale messages per note (16 pitches, 5 durations)
T1	law of one voice (pitch leads)	pitch, duration and joint deviations within sampling error: every $ z \leq 3.5$; no off-machine transition
T2	harmony by independence	five schedulers, byte-identical performances
T3	two-sided fitting	max : first resolution emits $\min(m, n)$ notes; one : 1, then $\min(m, n) - 1$ more
T4	instrument as a score	pitch = hex digits of π , duration = base-5 digits of e , one digit each, 400 notes
T5	law of the dual (rhythm leads)	as T1, via <code>COMM₂</code>
T6	simultaneity by contention	two notes per resolution; equal durations never
T7	style $\otimes$ physicality	transitions match normalised product; $ z \leq 3.5$; no forbidden transition
T8	idioms from the past	zero triple repeats under the crisp factor; no silence; motif completion within 0.03 of analytic
T9	table = formula	zero difference over 3000 random views (exact rationals)
T10	reflective = step semantics	identical performance over 3000 notes; exactly two null notes per note, all (r, ϵ)
T11	reflective voice, inertness	at most one record location with a candidate; $ \Pi - 1$ dead hands per note
T12	Twinkle	22 notes, end 4, identical under both orders

T10 is the central test. In the simulator it compares the reflective voice against a Python-constant voice. Here both sides are scores: the reflective `Voice` from `std.score` against an unrolled voice of k written steps, with the same clause and the same musical stream.

10.2 Additional tests

- **Congruence.** Normal-form interning agrees with an independent decision procedure on 10^4 random term pairs, including `*@` and α -variants.
- **Cross-implementation differential.** For T0–T12 scores and random small scores, the Rust player and `skeinsim.py`, driven by the same recorded choice indices, emit the same note sequence. The Python side gains a small adapter that accepts a choice list; this is the faithfulness obligation, kept as a permanent CI job.

- **Replay.** Every conformance run’s trace replays to an identical performance.
- **Enforcement.** A window-behavioural score using Viable_β (Proposition 6.3) runs 10^4 notes without violating β and without falling silent; the same score without the viability factor is shown, on a fixture, to fall silent.
- **Chords.** A chord receipt emits k notes at one onset, and its continuation is stamped at the longest duration.
- **Freshness guard.** A deliberately non-fresh score run with `--gc freshness` fails with `freshness-violated`.
- **MIDI.** Twinkle’s file is byte-identical to a golden file.

11 Phasing

phase	content	exit criteria
P0	workspace; <code>score-core</code> arena, normal forms, substitution, digests; parser for the core syntax and alphabets; pretty-printer	round-trip parse/print; congruence property test; Twinkle parses
P1	engine with clause-free receipts: soup, candidates, contention, maximal matchings, time, schedulers; performance output; trace	T3, T6, T12
P2	local clauses: atoms, spatial formulae, tables, machines, exact weights; PRNG source with interval decoding; macros and <code>par</code> comprehensions	T0, T1, T5, T7, T8, T9
P3	<code>std.score</code> : reflective voice, handoff, hocket; freshness collection; stream keys	T2, T10, T11; freshness guard
P4	MIDI writer; spigot sources and factor routing; replay; live MIDI	T4; replay; MIDI golden
P5	window-behavioural fragment: <code><></code> , <code>leads</code> , ν ; chord receipts	enforcement and chord tests
P6	human chooser; wasm build of the core crates; cross-implementation CI	differential job green; wasm build in CI

P0–P3 deliver a player for every example in the note except Proposition 6.3 and Definition 5.5; P4 makes it musically usable; P5 completes the calculus as the note defines it.

12 Decisions to confirm

1. **Home of the code.** Candidates: a new repository `F1R3FLY-io/f1r3score`; a directory beside the note in `publications/f1r3score`; or `F1R3Games`. Since the calculus is deliberately not F1R3Skein, the proposal is a new repository, with the note’s directory linking to it.
2. **Surface syntax.** Confirm Decision 3.1: a CFL dialect now, restated as **Theory Score** in the DDL later. In particular confirm the name syntax `<@P, tau, d>` and the file extension `.score`.

3. **Exact rationals by default.** Decision 7.1 and R5.2 put bit-for-bit reproducibility ahead of speed. An `f64` mode could be added for large ensembles; the proposal is not to add one until a measured need appears.
4. **Stream keys.** Decision 7.2 defaults to timbre, with optional labels. Two voices sharing a timbre (hocket) would otherwise share a stream; confirm that labels are the right remedy rather than, say, keying by base.
5. **Scheduler default.** `canonical` makes runs reproducible without a seed. Confirm, or prefer a seeded random default.
6. **Behavioural modalities.** Decision 5.5 evaluates them against the actual company, and R5.4 admits them only in the window fragment. Confirm that a bounded-lookahead approximation, which would not guarantee liveness, should stay out.
7. **Chord continuation.** The longest-duration convention of Definition 5.5 is isolated in one function; confirm it as the default while open thread 5 is pursued.
8. **Handoff record.** Proposition 6.5 notes that the outermost record of a handoff carries a pitch that was never played. Confirm that `last[...]` should not skip it automatically (the proposal: expose it, and give `std.score` a `call[...]` atom that skips it).
9. **Pitch naming.** Scientific pitch names with $C4 = 60$, and explicit MIDI numbers otherwise. Confirm, including enharmonic spellings as distinct data (so `F#4` and `Gb4` are different pitches with the same MIDI number).